



# JAVA HANDBOOK



1/10

## 1. What is Java?

Java is a high-level, class-based, object-oriented programming language developed by James Gosling at Sun Microsystems (1995).

It is designed to have as few implementation dependencies as possible.

## 3. JDK vs JRE vs JVM

| Term | Description                                                                   | Contains                                                 |
|------|-------------------------------------------------------------------------------|----------------------------------------------------------|
| JDK  | Java Development Kit. Used to develop Java applications.                      | JRE + Compiler (javac) Debugging tools, libs etc.        |
| JRE  | Java Runtime Environment. Used to run Java applications.                      | JVM + Core Libraries and other files                     |
| JVM  | Java Virtual Machine. Executes the bytecode and provides runtime environment. | Class Loader, Runtime Data Areas, Execution Engine, etc. |

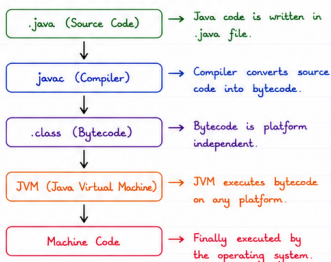
## 5. How Java Program Works?

1. Write code in .java file.
2. Compile using javac → Generates .class file (bytecode).
3. JVM loads the bytecode.
4. Bytecode is verified for security.
5. JVM interprets or JIT compiles to machine code.
6. OS executes the machine code.

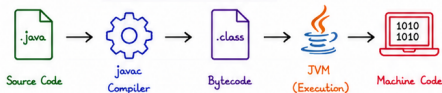
## 2. Features of Java

- Platform Independent - Write once, run anywhere
- Object-Oriented - Based on classes and objects
- Robust - Strong memory management & exceptions
- Secure - No explicit pointer, secure code execution
- Multithreaded - Supports concurrent programming
- Portable - Architecture neutral and portable
- High Performance - JIT compiler improves performance
- Distributed - Built-in support for networking
- Dynamic - Classes are loaded at runtime

## 4. Java Compilation Process



### Compilation Flow Diagram



### Interview Tip



- ★ Java is platform independent because JVM acts as an intermediate layer between the bytecode and the underlying operating system.



# JAVA HANDBOOK



2/10

## 1. Java Data Types

- Java is a strongly typed language.

### (A) Primitive Data Types

| Type    | Size    | Default Value | Range                   |
|---------|---------|---------------|-------------------------|
| byte    | 1 byte  | 0             | -128 to 127             |
| short   | 2 bytes | 0             | -32,768 to 32,767       |
| int     | 4 bytes | 0             | $-2^{31}$ to $2^{31}-1$ |
| long    | 8 bytes | 0L            | $-2^{63}$ to $2^{63}-1$ |
| float   | 4 bytes | 0.0f          | ~6-7 decimal digits     |
| double  | 8 bytes | 0.0d          | ~15 decimal digits      |
| char    | 2 bytes | '\u0000'      | 0 to 65,535             |
| boolean | 1 bit   | false         | true or false           |

### (B) Non-Primitive (Reference) Types

Classes, Interfaces, Arrays, Strings, Collections, etc.

## 2. Variables in Java

- A variable is a container that holds data.
- In Java, every variable must be declared before use.

Syntax:

```

dataType  variableName = value;
  ↓         ↓           ↓
  Type      Name      Value
    
```

Example:

```

int age = 20;
double salary = 45000.50;
char grade = 'A';
boolean isJavaFun = true;
    
```

## 3. Type Casting

### (A) Widening (Implicit)

- Automatically done by the compiler.
- From smaller type to larger type.

```

int i = 10;
double d = i; // int → double
    
```

### (B) Narrowing (Explicit)

- Manual casting is required.
- From larger type to smaller type.

```

double d = 99.99;
int i = (int) d; // double → int
    
```

## 4. Operators in Java

- Operators are used to perform operations on variables and values.
- Types: Arithmetic, Relational, Logical, Assignment, etc.

| Type       | Operators       | Example       | Description           |
|------------|-----------------|---------------|-----------------------|
| Arithmetic | + - * / %       | a + b         | Basic math operations |
| Relational | == != > < >= <= | a > b         | Compare values        |
| Logical    | &&    !         | a && b        | Logical operations    |
| Assignment | = += -= *= /=   | a += 5        | Assign and update     |
| Unary      | ++ -- + - !     | ++a           | Increment/Decrement   |
| Ternary    | ?:              | a > b ? a : b | Conditional operator  |
| Bitwise    | &   ^ ~ << >>   | a & b         | Bitwise operations    |

## 5. Control Statements

- Control statements are used to control the flow of execution.
- Types: if, switch, for, while, do-while, break, continue.

```

if (age >= 18) {
    System.out.println("Adult");
} else {
    System.out.println("Minor");
}
    
```

```

for (int i = 1; i <= 5; i++) {
    System.out.println(i);
}
    
```

## 6. String in Java

- String is a sequence of characters.
- In Java, String objects are immutable.

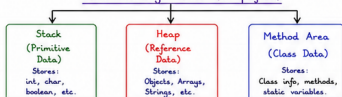
```

String name = "Java";
String s2 = new String("Hello");
System.out.println(name.length());
System.out.println(name.toUpperCase());
    
```

Strings are immutable in Java!



### Java Memory Model (Simplified)



### Interview Tip

- ★ Java is Platform Independent.
- ★ "Write Once, Run Anywhere" is the power of Java.
- ★ Focus on fundamentals - OOP, Collections, Multithreading, Exception Handling.
- ★ Practice + Consistency = Success 😊





# JAVA HANDBOOK



3/10

## 1. Object-Oriented Programming (OOP)

- Java is an object-oriented programming language.
- It is based on the concept of classes and objects.
- OOP makes code modular, reusable, and easy to maintain.

Class → Blueprint or template

Object → Instance of a class

Example → Class: Car

Object: myCar (a specific car)

## 2. OOPs Concepts in Java

|               |                                                                                                                                                                                     |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Class         | <ul style="list-style-type: none"><li>• A class is a blueprint for creating objects.</li><li>• It contains fields (data) and methods (functions).</li></ul>                         |
| Object        | <ul style="list-style-type: none"><li>• An object is an instance of a class.</li><li>• It has state (data) and behavior (methods).</li></ul>                                        |
| Encapsulation | <ul style="list-style-type: none"><li>• Wrapping data and methods into a single unit.</li><li>• Achieved using access modifiers (private, public, protected).</li></ul>             |
| Inheritance   | <ul style="list-style-type: none"><li>• One class can inherit fields and methods of another class.</li><li>• Promotes code reusability.</li></ul>                                   |
| Polymorphism  | <ul style="list-style-type: none"><li>• Ability of an object to take many forms.</li><li>• Achieved by Method Overloading (compile-time) and Method Overriding (runtime).</li></ul> |
| Abstraction   | <ul style="list-style-type: none"><li>• Hiding implementation details and showing only functionality.</li><li>• Achieved using abstract classes and interfaces.</li></ul>           |

## 3. Class & Object Example

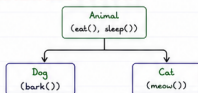
```
class Student {
    int id;
    String name;
    void display() {
        System.out.println("ID: " + id + ", Name: " + name);
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student(); // object creation
        s1.id = 101; // setting values
        s1.name = "Alice";
        s1.display(); // calling method
    }
}
```

Output:

ID: 101, Name: Alice

## 4. Inheritance Example



```
class Animal {
    void eat() { System.out.println("eat"); }
    void sleep() { System.out.println("sleep"); }
}

class Dog extends Animal {
    void bark() { System.out.println("bark"); }
}

class Cat extends Animal {
    void meow() { System.out.println("meow"); }
}
```

- Dog and Cat are subclasses of Animal.
- They inherit eat() and sleep() methods.
- They can have their own methods as well.

Example Usage:

```
Dog d = new Dog();
d.eat(); // inherited
d.sleep(); // inherited
d.bark(); // own method
```

## 5. Access Modifiers

| Modifier              | Within Class | Within Package | Outside Package (Subclass) | Outside Package (Non-Subclass) |
|-----------------------|--------------|----------------|----------------------------|--------------------------------|
| private               | Yes          | No             | No                         | No                             |
| default (no modifier) | Yes          | Yes            | No                         | No                             |
| protected             | Yes          | Yes            | Yes                        | No                             |
| public                | Yes          | Yes            | Yes                        | Yes                            |

- **private** - Accessible only within the same class.
- **default** - Accessible within the same package.
- **protected** - Accessible within the same package and by subclasses.
- **public** - Accessible from anywhere.

## 6. this Keyword

- 'this' refers to the current object.
- Used to access current class members.
- Used to call current class constructors.
- Helps in avoiding naming ambiguity.

Why use 'this'?

- ★ Improves code readability.
- ★ Resolves ambiguity between local variables and instance variables.
- ★ Can be used to pass current object as argument.

```
class Student {
    int id;
    String name;
    Student(int id, String name) {
        this.id = id; // refers to current object
        this.name = name;
    }
    void show() {
        System.out.println(this.id + " " + this.name);
    }
}
```

### OOPs Benefits

- ✓ Code Reusability
- ✓ Easy Maintenance
- ✓ Scalability
- ✓ Data Security
- ✓ Real-World Modeling



### Real Life Analogy

- Class → Blueprint of a House
- Object → Actual House built using the Blueprint
- Methods → Actions (open door, close window)
- Data Members → House properties (color, size)



### Interview Tip

- ★ OOP is the foundation of Java.
- ★ Think in terms of objects in real-world scenarios.





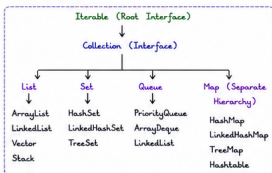
# JAVA HANDBOOK



4/10

## 1. Java Collections Framework

- Collections in Java provide a way to store and manipulate groups of objects efficiently.
- It is part of java.util package.
- It provides data structures like List, Set, Map, Queue and their implementations.
- The framework is designed with interfaces and classes.



## 2. Collections Interfaces Comparison

| Feature         | List                                   | Set                                       | Queue                                                        | Map                                 |
|-----------------|----------------------------------------|-------------------------------------------|--------------------------------------------------------------|-------------------------------------|
| Definition      | Ordered collection (allows duplicates) | Collection that does not allow duplicates | Collection designed for holding elements prior to processing | Collection of key-value pairs       |
| Duplicates      | Yes                                    | No                                        | Yes                                                          | Keys: No<br>Values: Yes             |
| Null Elements   | Yes                                    | At most one                               | Yes                                                          | Key: No<br>Value: Yes               |
| Order           | Insertion Order                        | No Order                                  | FIFO Order                                                   | No Order                            |
| Access          | By index                               | No index                                  | Head / Tail                                                  | By key                              |
| Interface       | List<E>                                | Set<E>                                    | Queue<E>                                                     | Map<K,V>                            |
| Example Classes | ArrayList<br>LinkedList<br>Vector      | HashSet<br>LinkedHashSet<br>TreeSet       | PriorityQueue<br>ArrayDeque<br>LinkedList                    | HashMap<br>LinkedHashMap<br>TreeMap |

## 3. Common Implementations

### (A) ArrayList (List)

- Resizable array implementation.
- Maintains insertion order.
- Allows duplicates and multiple nulls.
- Fast random access (get/set).
- Slower on insert/delete in middle.

**Example:**  
`List<String> list = new ArrayList<>();`  
`list.add("A");`  
`list.add("B");`  
`list.get(0);` // A  
`list.remove(1);` // removes B

### (B) LinkedList (List, Queue, Deque)

- Doubly-linked list.
- Maintains insertion order.
- Allows duplicates and multiple nulls.
- Fast insert/delete.
- Slower random access.

**Example:**  
`LinkedList<Integer> list =`  
`new LinkedList<>();`  
`list.add(1);`  
`list.addFirst(0);`  
`list.addLast(2);`

### (C) HashSet (Set)

- Hash table based.
- Does not allow duplicates.
- At most one null.
- No guarantee of order.
- Best performance for search operations.

**Example:**  
`Set<String> set = new HashSet<>();`  
`set.add("A");`  
`set.add("B");`  
`set.add("A");` // ignored  
`set.contains("B");` // true

### (D) TreeSet (Set)

- Red-Black tree based.
- Does not allow duplicates.
- Elements are sorted in natural order.
- Slower than HashSet.

**Example:**  
`Set<Integer> set = new TreeSet<>();`  
`set.add(30);`  
`set.add(10);`  
`set.add(20);`  
// Output: [10, 20, 30]

### (E) HashMap (Map)

- Hash table based.
- Stores key-value pairs.
- Keys are unique.
- Allows one null key and multiple null values.
- No order guarantee.

**Example:**  
`Map<Integer, String> map =`  
`new HashMap<>();`  
`map.put(1, "One");`  
`map.put(2, "Two");`  
`map.get(1);` // One

### (F) LinkedHashMap (Map)

- Hash table + Linked list.
- Maintains insertion order.
- Keys are unique.
- Allows one null key.

**Example:**  
`Map<Integer, String> map =`  
`new LinkedHashMap<>();`  
`map.put(1, "One");`  
`map.put(2, "Two");`  
// Iterates in insertion order

### (G) TreeMap (Map)

- Red-Black tree based.
- Keys are sorted.
- Keys must be comparable.
- No null keys.

**Example:**  
`Map<Integer, String> map =`  
`new TreeMap<>();`  
`map.put(30, "Thirty");`  
`map.put(10, "Ten");`  
// Keys sorted: [10, 30]

### (H) PriorityQueue (Queue)

- Priority queue based on heap.
- Orders elements by priority (natural order or comparator).
- Head is the highest priority element.

**Example:**  
`Queue<Integer> pq = new PriorityQueue<>();`  
`pq.add(30); pq.add(10); pq.add(20);`  
`pq.poll();` // returns 10

## 4. Key Points

- ✓ Use List when you need order and duplicates.
- ✓ Use Set when you need unique elements.
- ✓ Use Map when you need key-value association.
- ✓ Choose implementation based on performance needs.

### Interview Tip

- ★ ArrayList for fast access.
- ★ LinkedList for frequent insert/delete.
- ★ HashSet for fast search.
- ★ TreeSet/TreeMap for sorted data.
- ★ HashMap for general purpose key-value.



### Collections Hierarchy (Summary)





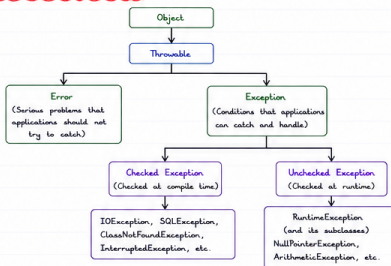
## 1. What is Exception?

- An exception is an event that occurs during the execution of a program that disrupts the normal flow of instructions.
- Java provides a robust and object-oriented way to handle runtime errors, called **Exception Handling**.

### Example:

```
int a = 10, b = 0;
int c = a / b; // ArithmeticException
System.out.println(c); // Never executed
```

## 2. Exception Hierarchy



## 3. Types of Exceptions

- Checked Exceptions** - Checked at compile time. Must be handled using try-catch or declared using throws.
- Unchecked Exceptions** - Checked at runtime. Subclasses of RuntimeException. Not mandatory to handle.
- Errors** - Serious issues like OutOfMemoryError, StackOverflowError, VirtualMachineError, etc. Applications should not try to handle.

### Key Point:

Handle exceptions you can recover from.  
Let the system handle Errors.

## 4. try-catch-finally

```
try {
    // risky code
} catch (Exception e) {
    // handle exception
} finally {
    // always executed
    // (cleanup code)
}
```

- try** - Contains code that may throw exception.
- catch** - Handles the exception.
- finally** - Always executes whether exception occurs or not. Used for cleanup (close files, db connections).

### Example:

```
FileInputStream fis = null;
try {
    fis = new FileInputStream("file.txt");
    int data = fis.read();
} catch (Exception e) {
    System.out.println("Error: " + e.getMessage());
} finally {
    try { if (fis != null) fis.close(); } catch (IOException ex) {}
}
```

## 5. throw vs throws

| throw                                   | throws                                         |
|-----------------------------------------|------------------------------------------------|
| Used to explicitly throw an exception.  | Used in method signature to declare exception. |
| Used inside method body.                | Used in method declaration.                    |
| Can throw only one exception at a time. | Can declare multiple exceptions.               |
| Used with any Throwable object.         | Used only with exception classes.              |

### Example:

```
if (age < 18) {
    throw new ArithmeticException("Not eligible");
}

void read() throws IOException, SQLException { ... }
```

## 6. Custom Exceptions

- We can create our own exception by extending Exception or RuntimeException.
- Useful for application-specific error conditions.

```
class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException(String message) {
        super(message);
    }
}

// Using the custom exception
void withdraw(double amount) throws InsufficientBalanceException {
    if (amount > balance) {
        throw new InsufficientBalanceException("Insufficient Balance");
    }
    // withdrawal logic
}
```

## 7. Best Practices

- Handle exceptions gracefully and provide meaningful messages.
- Catch specific exceptions, not generic Exception.
- Use finally (or try-with-resources) for cleanup.
- Do not ignore exceptions. Log them properly.
- Create custom exceptions for business logic.
- Avoid using exceptions for normal control flow.

### Example (try-with-resources):

```
try (BufferedReader br = new BufferedReader(new FileReader("file.txt"))) {
    String line = br.readLine();
    // use resource
} catch (IOException e) {
    System.out.println("Error: " + e.getMessage());
} // resource automatically closed
```

**try-with-resources**  
(auto closeable)  
Introduced in Java 7.  
Automatically closes resources.

## 8. Common Built-in Exceptions

| Exception                      | Type      | Description                            | Example                  |
|--------------------------------|-----------|----------------------------------------|--------------------------|
| ArithmeticException            | Unchecked | Arithmetic errors (divide by zero)     | int a = 10/0;            |
| NullPointerException           | Unchecked | Using null where object is expected    | str.length();            |
| ArrayIndexOutOfBoundsException | Unchecked | Invalid index of array                 | arr[5];                  |
| NumberFormatException          | Unchecked | Invalid conversion of string to number | Integer.parseInt("abc"); |
| IOException                    | Checked   | Input/Output related errors            | File read/write          |
| ClassNotFoundException         | Checked   | Class not found at runtime             | Class.forName("X");      |
| SQLException                   | Checked   | Database access errors                 | DB operations            |
| InterruptedException           | Checked   | A thread is interrupted                | Thread.sleep(1000);      |



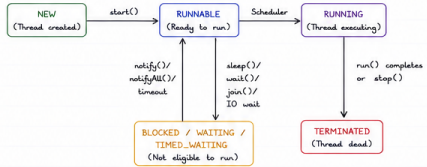
## 1. What is Multithreading?

- Multithreading is a feature that allows a program to execute multiple threads concurrently.
- Each thread is a lightweight sub-process.
- Threads share the same memory (heap) but have separate call stack.
- It improves responsiveness, throughput and CPU utilization.

### Benefits:

- ✓ Better performance
- ✓ Resource sharing
- ✓ Responsiveness
- ✓ Economy (less overhead than processes)

## 2. Thread Life Cycle



## 3. Creating Threads

### (A) By extending Thread class

```

class MyThread extends Thread {
    public void run() {
        System.out.println("Thread is running");
    }
}

MyThread t = new MyThread();
t.start(); // start() calls run() internally
  
```

### (B) By implementing Runnable interface

```

class MyRunnable implements Runnable {
    public void run() {
        System.out.println("Thread is running");
    }
}

Thread t = new Thread(new MyRunnable());
t.start();
  
```

### (C) Using Lambda Expression (Runnable)

```

Runnable r = () -> System.out.println("Thread is running");
Thread t = new Thread(r);
t.start();
  
```

## 4. Important Thread Methods

| Method             | Description                                         | Example                   |
|--------------------|-----------------------------------------------------|---------------------------|
| start()            | Starts the thread and calls run().                  | t.start();                |
| run()              | Contains the code that runs in the thread.          | public void run() { ... } |
| sleep(long ms)     | Pauses the thread for specified time.               | Thread.sleep(1000);       |
| join()             | Waits for this thread to die.                       | t.join();                 |
| yield()            | Suggests scheduler to give chance to other threads. | Thread.yield();           |
| isAlive()          | Checks if thread is alive.                          | t.isAlive();              |
| setPriority(int p) | Sets priority (1 to 10).                            | t.setPriority(7);         |
| getPriority()      | Returns priority of thread.                         | t.getPriority();          |
| interrupt()        | Interrupts the thread.                              | t.interrupt();            |
| isInterrupted()    | Checks if thread is interrupted.                    | t.isInterrupted();        |

## 5. Synchronization

- When multiple threads access shared data, inconsistency may occur.
- Synchronization ensures that only one thread accesses the shared resource at a time.
- It prevents race conditions.

### Example: Synchronized Method

```

class Counter {
    private int count = 0;
    public synchronized void increment() {
        count++;
    }
    public int getCount() { return count; }
}
  
```

### Key Point

Use synchronized when threads access shared data.  
It locks the object or block to allow only one thread.



## 6. Inter-Thread Communication

- Threads can communicate with each other using wait(), notify() and notifyAll() methods.
- These methods are defined in Object class.

### Producer (Thread A)

```

synchronized (lock) {
    // produce data
    lock.notify(); // or notifyAll()
}
  
```

### Consumer (Thread B)

```

synchronized (lock) {
    lock.wait(); // waits
    // consume data
}
  
```

- wait() - Releases the lock and waits.
  - notify() - Wakes up one waiting thread.
  - notifyAll() - Wakes up all waiting threads.
- (These methods must be called inside synchronized block.)

## 7. Daemon vs User Thread

| User Thread                           | Daemon Thread                                 |
|---------------------------------------|-----------------------------------------------|
| Runs in foreground.                   | Runs in background.                           |
| JVM waits for user threads to finish. | JVM does not wait for daemon threads.         |
| Created by default.                   | Must be set explicitly using setDaemon(true). |
| Example: main thread                  | Example: Garbage Collector thread             |

### Example

```

Thread t = new Thread(() -> { /* task */ });
t.setDaemon(true); // must be called before start()
t.start();
  
```

## 8. Thread Pools (Executor Framework)

- Creating a thread is costly.
- Thread pools reuse existing threads to execute tasks.
- Part of java.util.concurrent package.

```

ExecutorService executor = Executors.newFixedThreadPool(3);
executor.submit(() -> System.out.println("Task 1"));
executor.submit(() -> System.out.println("Task 2"));
executor.shutdown(); // stop accepting new tasks
executor.awaitTermination(5, TimeUnit.SECONDS);
  
```

### Common Executors

- newFixedThreadPool(n)
- newCachedThreadPool()
- newSingleThreadExecutor()
- newScheduledThreadPool(n)

### Interview Tip

- ★ Prefer Executor framework over creating threads directly.
- ★ Always shutdown executor to release resources.



### Quick Summary



### Key Points

- ✓ Threads are lightweight and share memory.
- ✓ Use synchronization to prevent conflicts.
- ✓ Thread life cycle: New → Runnable → Running → Blocked/Waiting → Terminated.

- ✓ Use wait()/notify() for communication.
- ✓ Daemon threads run in background.
- ✓ Thread pools improve performance and resource utilization.

Multithreading is the key to building high performance and responsive Java applications!





## 1. Introduction to Java 8

- Java 8 (released in March 2014) is a major update that brought functional programming features.
- It improves productivity, reduces boilerplate code and introduces new APIs.
- Java 8 is the most widely adopted version.

### Key Highlights

- ✓ Lambda Expressions
- ✓ Functional Interfaces
- ✓ Stream API
- ✓ Default & Static Methods in Interfaces
- ✓ Optional Class
- ✓ Method References
- ✓ Constructor References
- ✓ Date & Time API (java.time)
- ✓ Nashorn JavaScript Engine
- ✓ Parallel Array Operations

## 3. Functional Interfaces

- Functional interface has exactly one abstract method.
- It can have multiple default or static methods.
- Annotated with `@FunctionalInterface` (optional).

| Interface     | Abstract Method   | Description       | Example                     |
|---------------|-------------------|-------------------|-----------------------------|
| Runnable      | void run()        | Represents a task | <code>() → { ... }</code>   |
| Supplier<T>   | T get()           | Supplies a value  | <code>() → 10</code>        |
| Consumer<T>   | void accept(T t)  | Consumes a value  | <code>t → println(t)</code> |
| Function<T,R> | R apply(T t)      | Function mapping  | <code>x → x * x</code>      |
| Predicate<T>  | boolean test(T t) | Tests a condition | <code>x → x &gt; 0</code>   |

### Example:

```
Predicate<Integer> isEven = n → n % 2 == 0;
System.out.println(isEven.test(10)); // true
```

## 2. Lambda Expressions

- A lambda expression is a short block of code that takes parameters and returns a value.
- It is used to implement functional interfaces (single abstract method).

### Syntax:

```
(parameters) → expression
(parameters) → { statements; }
```

### Example:

```
(a) Without Lambda
Runnable r1 = new Runnable() {
    public void run() { System.out.println("Running..."); }
};

(b) With Lambda
Runnable r2 = () → System.out.println("Running...");
```

### Key Point

Lambda makes code concise, readable and expressive.



## 4. Method References

- Method reference is a shorthand notation of lambda expression.
- It improves readability.

| Type            | Syntax                | Example             | Equivalent Lambda                         |
|-----------------|-----------------------|---------------------|-------------------------------------------|
| Static Method   | ClassName::methodName | Math::abs           | <code>x → Math.abs(x)</code>              |
| Instance Method | object::methodName    | System.out::println | <code>x → System.out.println(x)</code>    |
| Instance Method | ClassName::methodName | String::length      | <code>s → s.length()</code>               |
| Constructor     | ClassName::new        | ArrayList::new      | <code>() → new ArrayList&lt;&gt;()</code> |

### Example:

```
List<String> names = Arrays.asList("A", "B", "C");
names.forEach(System.out::println);
```

## 5. Stream API

- Stream API provides a functional approach to process collections.
- Streams don't store data; they process data on-the-fly.
- Supports operations like filter, map, sort, reduce, collect, etc.

### Example:

```
List<Integer> nums = Arrays.asList(1,2,3,4,5,6);
List<Integer> evenSquares = nums.stream()
    .filter(n → n % 2 == 0) // filter even numbers
    .map(n → n * n) // square them
    .collect(Collectors.toList());
System.out.println(evenSquares); // [4, 16, 36]
```

### Stream Pipeline



## 6. Default & Static Methods in Interfaces

- Interfaces can have default and static methods with a body.
- Improves backward compatibility.

```
interface MyInterface {
    void show(); // abstract method
    default void greet() { // default method
        System.out.println("Hello from default method");
    }
    static void info() { // static method
        System.out.println("Static method in interface");
    }
}
```

### Example:

```
class MyClass implements MyInterface {
    public void show() { System.out.println("Show method"); }
}
MyInterface.info(); // call static method
```

### Key Point

Default methods are inherited by implementing classes.



## 7. Optional Class

- Optional is a container object which may or may not contain a non-null value.
- Helps to handle `NullPointerException` in a better way.
- Located in `java.util` package.

### Example:

```
Optional<String> name = Optional.ofNullable(getName());
if(name.isPresent()) {
    System.out.println(name.get());
} else {
    System.out.println("Name not available");
}
```

### Common Methods

- `of(value)` - create Optional
- `ofNullable(value)` - allow null
- `isPresent()` - check value
- `get()` - get value
- `orElse(default)` - default value
- `orElseGet(() → value)`
- `orElseThrow(() → exception)`

## 8. Date & Time API (java.time)

- New Date & Time API is immutable, thread-safe and easy to use.
- Package: `java.time`

| Class             | Purpose                      | Example                                                |
|-------------------|------------------------------|--------------------------------------------------------|
| LocalDate         | Represents date (yyyy-MM-dd) | <code>LocalDate.now()</code>                           |
| LocalTime         | Represents time (HH:mm:ss)   | <code>LocalTime.now()</code>                           |
| LocalDateTime     | Represents date & time       | <code>LocalDateTime.now()</code>                       |
| DateTimeFormatter | Formats date/time            | <code>DateTimeFormatter.ofPattern("dd-MM-yyyy")</code> |
| Period            | Period between two dates     | <code>Period.between(d1, d2)</code>                    |
| Duration          | Duration between two times   | <code>Duration.between(t1, t2)</code>                  |

### Example:

```
LocalDate today = LocalDate.now();
DateTimeFormatter fmt = DateTimeFormatter.ofPattern("dd/MM/yyyy");
System.out.println(today.format(fmt)); // e.g., 25/05/2024
```

## 9. Other Notable Features

- Nashorn JavaScript Engine - Execute JavaScript in JVM.
- Parallel Array Operations - `Arrays.parallelSort()`, `parallelPrefix()`, etc.
- Base64 Encoding/Decoding - `java.util.Base64`
- New Collectors - `joining()`, `groupingBy()`, `partitioningBy()`, etc.
- Improved Type Inference - Compiler can infer more types.
- Resolving Ambiguities - Some annotation multiple times.
- Parameter Names in Reflection - Retain parameter names.
- CompletableFuture - For async programming.
- Supports non-blocking, asynchronous operations.

```
CompletableFuture.supplyAsync(() → "Hello")
    .thenApply(msg → msg + " Java 8")
    .thenAccept(System.out::println);
```



## 10. Java 8 Interview Tips

- ★ What is difference between Java 7 and Java 8?
- ★ Explain Lambda expression with example.
- ★ What are functional interfaces? Give examples.
- ★ What is Stream API? How does it work?
- ★ Difference between `map()` & `flatMap()`?
- ★ What is Optional? Why is it used?
- ★ Explain default and static methods in interfaces.
- ★ What are method references? Give types.
- ★ What is the new Date & Time API?
- ★ What is CompletableFuture?



Java 8 made Java more powerful, expressive and developer friendly!





## 1. Stack vs Heap Memory

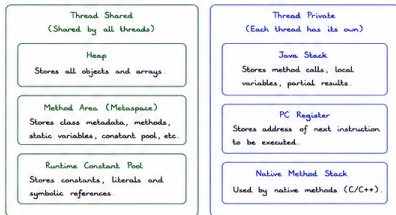
- Java has two major memory regions for storing data - Stack and Heap.

| Stack Memory                                                    | Heap Memory                                              |
|-----------------------------------------------------------------|----------------------------------------------------------|
| Used for method calls, local variables, primitives, references. | Used for objects and arrays (created using new keyword). |
| Fixed size (small).                                             | Dynamically allocated (larger).                          |
| Follows LIFO order.                                             | No specific order.                                       |
| Memory automatically released when method returns.              | Garbage Collector reclaims unused objects.               |
| Faster access.                                                  | Relatively slower.                                       |

### Key Point:

Objects are created in heap but references are stored in stack.

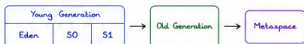
## 2. JVM Memory Areas (Runtime Data Areas)



**Note:** Heap, Method Area and Runtime Constant Pool are shared. Stack, PC Register and Native Method Stack are private to each thread.

## 3. Young, Old & Metaspace

- Heap is divided into different regions for efficient GC.
- Young Generation** - For new objects.
  - Eden Space - New objects are created here.
  - Survivor Space 0 (S0) - From/To spaces.
  - Survivor Space 1 (S1) - Objects that survive GC move between S0 and S1.
- Old Generation (Tenured)** - Stores long-lived objects. Objects moved here after surviving multiple GCs.
- Metaspace** - Replaces PermGen (since Java 8). Stores class metadata. Located in native memory.



## 4. Garbage Collection (GC)

- GC automatically reclaims memory occupied by objects that are no longer referenced.
- Types of GC:

| Type               | When it Occurs                                        | Area                         | Purpose                                                  |
|--------------------|-------------------------------------------------------|------------------------------|----------------------------------------------------------|
| Minor GC           | When Eden space is full.                              | Young Generation             | Reclaims memory of unreferenced objects in Eden & S0/S1. |
| Major GC (Full GC) | When Old Generation is full or explicitly called.     | Old Generation (+ Metaspace) | Reclaims memory from Old Gen & does compaction.          |
| Mixed GC           | After Minor GC when some objects promoted to Old Gen. | Young + Old                  | Cleans young gen and reclaims some space in Old Gen.     |
| Concurrent GC      | Runs concurrently with application threads.           | Heap (both)                  | Improves performance by minimizing pauses.               |

Popular GC Algorithms: Serial, Parallel, CMS (Concurrent Mark Sweep), G1 (Garbage First), ZGC, Shenandoah.

## 5. String Pool

- String Pool is a special storage in heap (part of Metaspace in Java 8+) that stores string literals.
- If a string literal already exists in pool, same reference is returned.
- Helps in memory optimization.

**Example:**

```
String s1 = "hello";
String s2 = "hello";
String s3 = new String("hello"); // New object in heap

s1 == s2 → true
s1 == s3 → false
```

// Same reference (from pool)



## 6. Memory Leaks in Java

- Memory leak occurs when objects are no longer used but still referenced, so GC can't reclaim them.
- Common causes:
  - Unclosed resources (files, streams, connections).
  - Static collections holding references.
  - Listeners or callbacks not removed.
  - Inner classes holding reference to outer class.
  - Caches growing indefinitely.

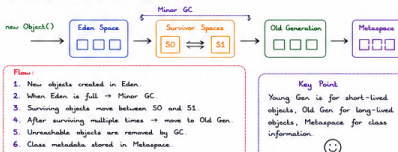
**Example:**

```
List<Object> list = new ArrayList<>();
while(true) {
    list.add(new Object());
    // Memory leak
}
```

## 7. OutOfMemoryError vs StackOverflowError

| OutOfMemoryError                                                                                                                                                                                                                                                                                             | StackOverflowError                                                                                                                                                                                                                             |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>Occurs when JVM runs out of heap memory.</li> <li>Too many objects created.</li> <li>GC unable to free enough memory.</li> <li>Types: Java heap space, GC overhead limit, etc.</li> <li>Fix: Reduce object creation, increase heap size, fix memory leaks.</li> </ul> | <ul style="list-style-type: none"> <li>Occurs when stack memory is exhausted.</li> <li>Infinite recursion or too deep method calls.</li> <li>Each call adds a stack frame.</li> <li>Fix: Fix recursion, increase stack size (-Xss).</li> </ul> |

## 8. JVM Memory Flow Diagram



## 9. Interview Tips & Best Practices

- Understand Stack, Heap and their differences.
- Know JVM memory areas and their purpose.
- Be clear about Young Gen, Old Gen and Metaspace.
- Understand Minor GC vs Major GC.
- Explain String Pool with example.

- Know common memory leaks and how to avoid.
- Understand OOME and StackOverflowError.
- Know JVM options: -Xms, -Xmx, -Xss, -XX:MaxMetaspaceSize.
- Use tools: jvisualvm, jconsole, eclipse MAT for analysis.
- Write memory efficient and clean code.

### JVM Options (Common)

- Xms: Initial heap size.
- Xmx: Maximum heap size.
- Xss: Thread stack size.
- XX:MaxMetaspaceSize: Max Metaspace size.
- verbose:gc: GC logs.



### Remember:

- JVM memory management & Garbage Collection = Better performance, scalability & stability.
- Write clean code, release resources and let GC do its job!



### Useful Tools

- jvisualvm
- jconsole
- Visual GC
- Eclipse MAT
- YourKit
- JProfiler

😊 Happy Coding!



# JAVA HANDBOOK



9/10

## 1. Java Interview Cheat Sheet (Core Java)

- Java is platform independent.
- JVM is part of JRE and JRE is part of JDK.
- "==" compares reference for objects.
- "equals()" compares content.
- String is immutable.
- StringBuilder is mutable and not synchronized.
- StringBuffer is mutable and synchronized.
- Array index starts from 0.
- Exception handling improves robustness.
- Collections framework provides ready-to-use data structures.
- HashMap allows one null key and multiple null values.

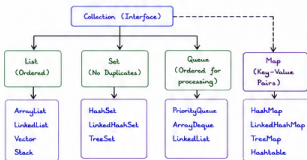
**Key Point**  
Practice coding and understand concepts deeply.  
Interview is about clarity and confidence!



## 2. Important Java Keywords

| Keyword      | Description                                        | Keyword    | Description                     |
|--------------|----------------------------------------------------|------------|---------------------------------|
| abstract     | Cannot be instantiated, can have abstract methods. | extends    | Used to inherit a class.        |
| final        | Cannot be changed/extended/overridden.             | implements | Used to implement an interface. |
| static       | Belongs to class, not to instance.                 | super      | Refers to parent class.         |
| synchronized | Used for thread synchronization.                   | this       | Refers to current object.       |
| transient    | Not serialized.                                    | package    | Groups related classes.         |
| volatile     | Value may change unexpectedly (threads).           | import     | Imports classes/packages.       |
| native       | Implemented in native code (non-Java).             | throws     | Declares exceptions.            |
| strictfp     | Ensures portable floating-point results.           | instanceof | Checks object type.             |

## 3. Collections Framework Overview



### Notes:

- List - Maintains insertion order, allows duplicates.
- Set - Does not allow duplicates (except some like TreeSet in libraries).
- Queue - For processing elements (FIFO or Priority).
- Map - Stores key-value pairs, keys are unique.

## 4. Commonly Used Classes

| Class            | Purpose                              | Example                                     |
|------------------|--------------------------------------|---------------------------------------------|
| Object           | Root class of all classes.           | Object obj = new Object();                  |
| String           | Immutable sequence of characters.    | String s = "hello";                         |
| StringBuilder    | Mutable sequence (not synchronized). | StringBuilder sb = new StringBuilder("hi"); |
| StringBuffer     | Mutable sequence (synchronized).     | StringBuffer sb = new StringBuffer("hi");   |
| Math             | Mathematical functions.              | Math.max(10, 20);                           |
| Arrays           | Utility methods for arrays.          | Arrays.sort(arr);                           |
| Scanner          | To read input from user.             | Scanner sc = new Scanner(System.in);        |
| System           | System utilities (in, out, err, gc). | System.out.println("Hi");                   |
| Date / LocalDate | Represents date & time.              | LocalDate now();                            |
| Random           | Generate random numbers.             | Random r = new Random();                    |

## 5. Wrapper Classes

- Primitive types are not objects.
- Wrapper classes convert primitive to object.
- Useful in Collections and when object is required.

| Primitive Type | Wrapper Class |
|----------------|---------------|
| int            | Integer       |
| char           | Character     |
| boolean        | Boolean       |
| double         | Double        |
| float          | Float         |
| long           | Long          |
| short          | Short         |
| byte           | Byte          |

### Example:

```

Integer i = 10; // autoboxing
int x = i; // unboxing
Boolean b = true;
Character c = 'A';
  
```

## 6. OOPs Concepts in Java

| Concept            | Description                                                         |
|--------------------|---------------------------------------------------------------------|
| Class              | Blueprint for objects.                                              |
| Object             | Instance of a class.                                                |
| Encapsulation      | Wrapping data and methods together, restricting access.             |
| Abstraction        | Hiding implementation details, showing essential features.          |
| Inheritance        | Child class inherits properties of parent class.                    |
| Polymorphism       | One interface, multiple implementations (overloading & overriding). |
| Method Overloading | Same method name, different parameters (compile-time).              |
| Method Overriding  | Same method name, same params (runtime).                            |
| Constructor        | Special method to initialize objects.                               |
| this / super       | this → current class reference, super → parent class reference.     |

## 7. Access Modifiers

| Modifier  | Same Class | Same Package | Sub Class (Different Package) | Different Package (Non Sub Class) |
|-----------|------------|--------------|-------------------------------|-----------------------------------|
| private   | Yes        | No           | No                            | No                                |
| default   | Yes        | Yes          | No                            | No                                |
| protected | Yes        | Yes          | Yes                           | No                                |
| public    | Yes        | Yes          | Yes                           | Yes                               |

### Notes:

default (no modifier) is visible within the same package only.

## 8. Java 8 Important Additions (Quick List)

- Lambda Expressions
- Functional Interfaces
- Stream API
- Default Methods in Interfaces
- Static Methods in Interfaces
- Method References
- Optional Class
- New Date & Time API (java.time)
- forEach() with Iterable
- Parallel Streams
- CompletableFuture (API for async programming)

## 9. Important Code Snippets

|                                                                                                          |                                                                                                                                                                      |                                                                                                                              |                                                                                                                                                                                       |                                                                                                                                                                                                  |
|----------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Swap Two Numbers</b><br><pre>int a = 10, b = 20; a = a + b; b = a - b; a = a - b; // a=20, b=10</pre> | <b>Check Palindrome String</b><br><pre>String s = "madam"; String rev = new StringBuilder(s)     .reverse().toString(); boolean isPal = s.equals(rev); // true</pre> | <b>Factorial of a Number</b><br><pre>int n = 5, fact = 1; for (int i = 1; i &lt;= n; i++)     fact *= i; // fact = 120</pre> | <b>Count Vowels in String</b><br><pre>String s = "Hello World"; long count = s.chars()     .filter(ch -&gt; "aeiouAEIOU"         .indexOf(ch) != -1)     .count(); // count = 3</pre> | <b>Fibonacci Series</b><br><pre>int n = 7; int a = 0, b = 1; for (int i = 0; i &lt; n; i++) {     System.out.print(a + " ");     int next = a + b;     a = b; b = next; } // 0 1 1 2 3 5 8</pre> |
|----------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 10. Interview Tips

- Understand concepts, don't just memorize.
- Solve coding problems regularly.
- Explain your code clearly.
- Think before coding.
- Practice edge cases.
- Stay calm and confident.
- Ask questions if not clear.
- Revise core Java and OOPs.

### Common Mistakes

- Not handling nulls.
- Ignoring exception handling.
- Hardcoding values.
- Not considering time/space complexity.
- Not testing edge cases.

### Preparation Plan

- ✓ Revise Core Java
- ✓ Practice Collections & Streams
- ✓ SolveDSA Problems
- ✓ Build Small Projects
- ✓ Take Mock Interviews
- ✓ Analyse & Improve

### Remember

Java is about writing clean, efficient and maintainable code.  
Keep practicing and keep learning!



"The expert in anything was once a beginner."  
- Keep Coding, Keep Growing!





# JAVA HANDBOOK



10/10

## 1. Java Developer Roadmap



1

### Core Java Fundamentals

- Syntax, Variables, Data Types, Operators
- Control Flow, Methods, Arrays
- OOPs - Classes, Objects, Inheritance, Polymorphism, Abstraction, Encapsulation

Goal:

Build strong programming foundation

2

### Core Java Advanced

- Collections Framework
- Exception Handling
- Multithreading, Synchronization
- File I/O, Serialization, Annotations

Goal:

Write robust and efficient Java programs

3

### Java 8+ Features

- Lambda Expressions, Functional Interfaces
- Stream API, Optional Class
- Default & Static Methods
- Date & Time API, Method References

Goal:

Write clean, concise and modern code

4

### JVM, Memory & Performance

- JVM Architecture, Memory Areas
- Garbage Collection, String Pool
- Stack vs Heap, Class Loading
- JVM Options, Performance Tuning

Goal:

Understand internals and optimize apps

5

### Enterprise Java (Backend)

- JDBC, JPA / Hibernate
- Spring Framework (Core, MVC, Boot)
- RESTful Web Services, JSON, Swagger
- Spring Security, Validation

Goal:

Build scalable and secure backend apps

6

### Databases & SQL

- SQL Basics to Advanced (Joins, Subqueries)
- Normalization, Indexing
- Transactions, ACID Properties
- Database Design Best Practices

Goal:

Store and manage data efficiently

7

### Testing

- Unit Testing - JUnit, Mockito
- Integration Testing
- Test Driven Development (TDD)
- Code Coverage (JaCoCo)

Goal:

Write bug-free and reliable applications

8

### Build, Tools & DevOps Basics

- Maven / Gradle
- Git & GitHub
- Docker Basics
- CI/CD Basics (Jenkins / GitHub Actions)

Goal:

Automate build, deploy and collaborate

9

### System Design & Scalability

- System Design Basics
- Load Balancing, Caching, CDN
- Microservices Basics
- Design Patterns in Java

Goal:

Design scalable and high performance systems

10

### Advanced & Beyond

- Concurrency Utilities (Executor, CompletableFuture)
- Reactive Programming (Project Reactor)
- Kafka / Message Queues
- Cloud Platforms (AWS / Azure / GCP)

Goal:

Become job-ready and future-proof developer

★ Keep Learning, Keep Building, Keep Growing! 😊

## 6. Common Career Paths

- ★ Backend Developer - Work on server-side logic, APIs, databases
- ★ Full Stack Developer - Work on both frontend and backend
- ★ Software Engineer - Design, develop and maintain software
- ★ DevOps Engineer - Handle deployment, automation, infrastructure
- ★ QA / Automation Engineer - Test and ensure software quality
- ★ Solution Architect - Design scalable and robust systems
- ★ Tech Lead / Engineering Manager - Lead teams and drive projects

★ Remember: Your roadmap is unique. Keep exploring, keep building, and stay consistent! 😊



### Daily Learning Plan (Example)

- ✓ 1-2 hours: Learn a concept
- ✓ 1-2 hours: Practice coding problems
- ✓ 1 hour: Build / Work on project
- ✓ 30 mins: Read / Watch / Review

"The only way to learn a new programming language is by writing programs in it."

- Dennis Ritchie

## 7. Final Tips for Java Developers

- ★ Practice Coding Daily - Consistency beats intensity
- ★ Understand Core Concepts Deeply
- ★ Write Clean, Readable and Maintainable Code
- ★ Don't just copy - understand and build
- ★ Read others' code and write code reviews
- ★ Build real-world projects and solve problems
- ★ Take breaks, avoid burnout and enjoy the journey!

Golden Rule



Code. Learn. Build. Repeat.



Stay Connected &amp; Keep Growing!

- Be Curious
- Be Patient
- Be Consistent
- Never Stop Learning

All the Best! Happy Coding! ❤️

## 2. Best Resources for Learning



| Type           | Resources                                                     |
|----------------|---------------------------------------------------------------|
| Official Docs  | docs.oracle.com, dev.java                                     |
| Online Courses | Udemy, Coursera, Pluralsight, LinkedIn Learning               |
| Practice       | LeetCode, HackerRank, CodeChef, GeeksforGeeks                 |
| Books          | Head First Java, Effective Java, Java Concurrency in Practice |
| YouTube        | Telusko, Programming with Mosh, CodeWithHarry                 |
| Blogs          | Building, Java Code Geeks, DZone                              |
| Communities    | Stack Overflow, Reddit (r/java), Dev.to, Hashnode             |

## 3. Project Ideas (Practice & Portfolio)



1. Student Management System (CRUD with MySQL + Spring Boot)
2. E-Commerce REST API (Spring Boot, JWT, MySQL)
3. Blog Application (Spring Boot, JPA, Thymeleaf / React)
4. Online Book Store (Filter, Search, Cart, Orders)
5. Expense Tracker Application
6. Chat Application (WebSocket + Spring Boot)
7. URL Shortener Service
8. Task Management System (Kanban Style)
9. File Upload & Download Service (AWS S3)
10. Real-time Notification System (Kafka + Spring Boot)

Tip:

Build projects, share on GitHub and write good README! 😊

## 4. Essential Developer Tools

| Category         | Tools                           |
|------------------|---------------------------------|
| IDE              | IntelliJ IDEA, Eclipse, VS Code |
| Build Tools      | Maven, Gradle                   |
| Version Control  | Git, GitHub, GitLab, Bitbucket  |
| API Testing      | Postman, Insomnia               |
| Database         | MySQL, PostgreSQL, MongoDB, H2  |
| Testing          | JUnit, Mockito, TestNG          |
| Monitoring       | Prometheus, Grafana             |
| Collaboration    | Jira, Confluence, Slack         |
| Containerization | Docker, Kubernetes              |
| CI/CD            | Jenkins, GitHub Actions         |

Key Point

Good tools increase productivity and code quality. ★

## 5. Soft Skills for Success

- ✓ Problem Solving - Break problems into smaller parts
- ✓ Communication - Explain ideas clearly
- ✓ Teamwork - Collaborate and respect others
- ✓ Time Management - Prioritize and meet deadlines
- ✓ Continuous Learning - Stay updated with new tech
- ✓ Adaptability - Be open to change

